

## 개요

### 요약

이 문서는 CSCI103 Lecture 7의 강의 내용을 바탕으로 데이터 파이프라인을 효과적으로 운영하는 방법을 다룹니다. 데이터 과학자가 개발한 프로토타입을 신뢰할 수 있는 자동화된 파이프라인으로 전환하는 과정에 초점을 맞춥니다. 주요 내용은 요구사항 정의, 파이프라인 설계 원칙, 구성 요소, 데이터 유효성 검사 및 품질 관리, 거버넌스, 확장성 및 탄력성입니다. 특히 Databricks의 LakeFlow 및 선언적 파이프라인(Declarative Pipelines)을 활용하여 복잡한 운영 작업을 단순화하고 비용 효율성을 높이는 방법을 상세히 설명합니다.

용어 정리

데이터 파이프라인 운영화는 복잡한 개념들을 포함하므로, 핵심 용어들을 미리 이해하는 것이 중요합니다. 아래 표는 강의에서 다루는 주요 용어들을 쉽게 설명합니다.

Table 1: 주요 용어 정리

| 용어 (원어)                                         | 쉬운 설명                                                          | 비고                              |
|-------------------------------------------------|----------------------------------------------------------------|---------------------------------|
| 데이터 파이프라인 운영화 (Operationalizing Data Pipelines) | 데이터 과학자가 만든 프로토타입을 실제 비즈니스에서 지속적으로 사용할 수 있는 자동화된 시스템으로 만드는 과정. | 신뢰성, 자동화, 효율성 확보                |
| 기능적 요구사항 (Functional Requirements)              | 시스템이 '무엇을' 해야 하는지에 대한 비즈니스 관점의 요구사항.                           | 비즈니스 규칙, 인증, 보고서 등              |
| 비기능적 요구사항 (Non-functional Requirements)         | 시스템이 '어떻게' 작동해야 하는지에 대한 요구사항.                                  | 성능, 확장성, 보안, 가용성 등              |
| 데이터 사일로 (Data Silos)                            | 조직 내에서 데이터가 고립되어 다른 부서나 시스템에서 접근하기 어려운 상태.                     | 비효율성, 데이터 불일치 유발                |
| 데이터 거버넌스 (Data Governance)                      | 데이터의 품질, 보안, 접근성, 사용 정책 등을 관리하는 체계.                            | 데이터 신뢰성 및 규제 준수                 |
| 확장성 (Scalability)                               | 시스템이 추가 리소스를 통해 더 많은 작업을 처리할 수 있는 능력.                          | 수직적 (Scale Up), 수평적 (Scale Out) |
| 탄력성 (Elasticity)                                | 워크로드 변화에 따라 시스템 리소스를 자동으로 늘리거나 줄이는 능력.                         | 비용 효율성, 유연성                     |
| 신뢰성 (Reliability)                               | 시스템이 오류 없이 올바른 동작을 지속적으로 수행하는 능력.                              | 정확한 데이터 출력                      |
| 가용성 (Availability)                              | 시스템이 사용자가 필요할 때 언제든지 접근하여 사용할 수 있는 능력.                         | '5-나인즈' 등으로 표현                  |
| ETL (Extract, Transform, Load)                  | 데이터를 추출, 변환 후 로드하는 전통적인 방식.                                    | 스키마가 잘 정의된 경우 유리                |
| ELT (Extract, Load, Transform)                  | 데이터를 추출, 로드 후 변환하는 방식.                                         | 비정형/반정형 데이터에 유연                 |
| LakeFlow 선언적 파이프라인 (Declarative Pipelines)      | Databricks에서 제공하는, 데이터 파이프라인을 간단한 선언문으로 정의하는 방식.               | 복잡한 운영 로직 자동화                   |
| 스트리밍 테이블 (Streaming Tables)                     | 중분 데이터 처리를 위해 설계된 데이터.                                         | 한 번 처리된 데이터는 재처리 안 함            |
| 구체화된 뷰 (Materialized Views)                     | 미리 계산되어 캐시된 뷰로, 쿼리 성능을 향상시키고 비용을 절감.                           | 중분 업데이트 가능                      |
| 데이터 드리프트 (Data Drift)                           | 시간이 지남에 따라 데이터의 통계적 특성이 변하는 현상.                                | 모델 성능 저하 유발                     |

## 핵심 개념 · 원리

### 0.1 데이터 파이프라인 운영화의 이해

데이터 파이프라인 운영화는 데이터 과학자가 개발한 초기 프로토타입이나 개념 증명 (PoC) 을 실제 비즈니스 환경에서 지속적으로 가치를 창출할 수 있는 견고하고 자동화된 시스템으로 전환하는 과정입니다. 이는 단순한 코드 실행을 넘어, 데이터의 품질, 신뢰성, 보안, 확장성 등 다양한 비즈니스 및 기술적 요구사항을 충족시키는 것을 목표로 합니다.

#### 0.1.1 운영화의 필요성

데이터 과학자가 노트북에서 개발한 모델이나 분석 코드는 특정 시점의 데이터에 대한 통찰력을 제공할 수 있지만, 이를 실제 비즈니스 의사결정에 지속적으로 활용하기 위해서는 여러 단계를 거쳐야 합니다.

- **지속적인 결과 생산:** 비즈니스는 일회성 분석이 아닌, 최신 데이터를 기반으로 한 지속적인 통찰력을 필요로 합니다. 운영화된 파이프라인은 이를 자동화하여 제공합니다.
- **데이터 품질 및 신뢰성:** 하위 시스템(다운스트림 소비자)이 사용할 수 있는 깨끗하고 신뢰할 수 있는 데이터를 보장해야 합니다. 이는 대시보드 사용자, 다른 데이터 과학자, 머신러닝 모델 등에게 필수적입니다.
- **비즈니스 의사결정 지원:** 궁극적으로 운영화된 파이프라인은 비즈니스가 데이터를 기반으로 더 나은 의사결정을 내릴 수 있도록 돕습니다.

### 0.2 요구사항 정의: 파이프라인 설계의 기초

모든 시스템 설계의 첫 단계는 명확한 요구사항을 이해하는 것입니다. 데이터 파이프라인도 예외는 아닙니다. 요구사항은 시스템이 '무엇을' 해야 하고 '어떻게' 작동해야 하는지에 대한 청사진을 제공합니다.

#### 0.2.1 기능적 요구사항 (Functional Requirements)

기능적 요구사항은 시스템이 비즈니스 관점에서 '무엇을' 해야 하는지를 정의합니다. 이는 사용자가 시스템과 상호작용하여 수행할 수 있는 특정 기능과 동작에 초점을 맞춥니다.

- **비즈니스 규칙:** 특정 비즈니스 로직(예: 거래 데이터의 조정, 취소 처리 방식).
- **관리 기능:** 사용자 인증, 권한 부여 수준, 감사 추적.
- **외부 인터페이스:** 연결해야 할 데이터 소스(예: 여러 데이터베이스, API) 및 데이터 제공 대상, 데이터 형식.
- **인증 요구사항:** 데이터의 특정 표준 준수 여부.

- **보고 및 분석:** 생성해야 할 보고서 유형, 과거 데이터 사용 방식.

## 0.2.2 비기능적 요구사항 (Non-functional Requirements)

비기능적 요구사항은 시스템이 기능적 요구사항을 '어떻게' 수행해야 하는지를 정의합니다. 이는 시스템의 품질 속성, 성능, 제약 조건 등에 중점을 둡니다. 종종 간과되지만, 시스템의 성공에 기능적 요구사항만큼 중요합니다.

Table 2: 기능적 vs. 비기능적 요구사항 비교

| 구분  | 기능적 요구사항 (What)                                                                                                                    | 비기능적 요구사항 (How)                                                                                                                             |
|-----|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 정의  | 시스템이 수행해야 할 특정 기능 및 동작                                                                                                             | 시스템이 기능을 수행하는 방식의 품질 속성                                                                                                                     |
| 초점  | 비즈니스 로직, 사용자 상호작용                                                                                                                  | 성능, 보안, 확장성, 가용성, 유지보수성                                                                                                                     |
| 예시  | <ul style="list-style-type: none"> <li>- 고객 주문 처리</li> <li>- 재고 수준 업데이트</li> <li>- 월별 판매 보고서 생성</li> <li>- 사용자 로그인 및 인증</li> </ul> | <ul style="list-style-type: none"> <li>- 응답 시간 3초 이내</li> <li>- 연중무휴 99.999% 가용성</li> <li>- 초당 1000건의 트랜잭션 처리</li> <li>- 데이터 암호화</li> </ul> |
| 중요성 | 비즈니스 목표 달성                                                                                                                         | 사용자 만족도, 시스템 신뢰성, 비용 효율성                                                                                                                    |

- **성능 (Performance):**

- **지연 시간 (Latency):** 데이터가 시스템에 들어와 처리되어 결과가 나오는 데 걸리는 시간 (예: 5초, 10초, 밀리초).

- **처리량 (Throughput):** 단위 시간당 처리할 수 있는 데이터 양.

- **확장성 (Scalability):** 데이터 양이나 사용자 수가 증가할 때 시스템이 얼마나 잘 대응할 수 있는가.

- **용량 (Capacity):** 시스템이 처리할 수 있는 최대 데이터 양 또는 사용자 수.

- **가용성 (Availability):** 시스템이 사용 가능한 상태를 유지하는 시간의 비율.

- '5-나인즈 (99.999%)' 가용성은 연간 약 5분 정도의 다운타임만 허용함을 의미하며, 매우 높은 비용이 수반될 수 있습니다. 비즈니스 중요도에 따라 적절한 가용성 수준을 결정해야 합니다.

- **신뢰성 (Reliability):** 시스템이 일관되고 정확하게 작동하는 능력.

- **복구 가능성 (Recoverability):** 시스템 장애 발생 시 얼마나 빨리 정상 상태로 복구될 수 있는가.

- **유지보수성 (Maintainability):** 시스템을 얼마나 쉽게 수정, 개선, 확장할 수 있는가.

- **보안 (Security):** 데이터 및 시스템을 무단 접근, 사용, 공개, 파괴로부터 보호하는 능력.

- **관리 용이성 (Manageability):** 파이프라인을 얼마나 쉽게 모니터링하고 관리할 수 있는가.
- **데이터 무결성 (Data Integrity):** 데이터의 정확성, 일관성, 신뢰성을 유지하는 능력.
- **사용성 (Usability) 및 상호운용성 (Interoperability):** 시스템이 얼마나 사용하기 쉽고 다른 시스템과 잘 연동되는가.

비기능적 요구사항은 종종 간과되지만, 시스템의 성공에 결정적인 영향을 미칩니다. 예를 들어, 아무리 기능이 완벽해도 성능이 느리거나 자주 다운되는 시스템은 비즈니스에 가치를 제공하기 어렵습니다. 따라서 설계 전에 반드시 비즈니스 사용자에게 성능, 가용성 등에 대한 기대를 명확히 질문하고 이해해야 합니다. 과도한 성능은 불필요한 비용을 초래할 수 있습니다.

### 0.3 효과적인 데이터 파이프라인을 위한 원칙

데이터 파이프라인을 설계하고 운영할 때 고려해야 할 몇 가지 핵심 원칙들이 있습니다. 이 원칙들은 파이프라인의 효율성, 신뢰성, 가치 창출 능력을 극대화하는 데 도움을 줍니다.

#### 1. 데이터를 큐레이션하고 신뢰할 수 있는 데이터 제품으로 제공 (Curate data and offer trusted data as products):

- 파이프라인을 통해 생성되는 데이터를 단순한 출력물이 아닌, 비즈니스 사용자와 데이터 과학자가 소비할 수 있는 '제품'으로 간주해야 합니다.
- **멀티홉/메달리온 아키텍처 (Multihop/Medallion Architecture):** 원본 데이터 (Raw) -> 큐레이션된 데이터 (Curated) -> 비즈니스 준비 데이터 (Business Ready)의 3단계로 구성됩니다.
  - **Raw (Bronze) Layer:** 원본 데이터가 그대로 저장되는 계층.
  - **Curated (Silver) Layer:** 정제, 필터링, 보강된 데이터로, ML 모델 구축 등에 사용됩니다.
  - **Business Ready (Gold) Layer:** 집계되고 의미론적으로 정의된 데이터 마트 및 큐브 형태로, 비즈니스 사용자가 대시보드 등에 활용합니다.
- 각 홉(Hop)을 거칠수록 데이터 품질은 향상되지만, 그에 따른 비용도 증가합니다.

#### 2. 데이터 사일로 제거하고 데이터 이동을 최소화 (Remove data silos and minimize data movement):

- 데이터를 불필요하게 복제하거나 이동하는 것은 취약성, 품질 문제, 지연 시간 증가, 비용 상승을 초래합니다.
- **데이터 복제 방지:** 데이터 사일로 생성을 피하고, 불필요한 운영 의존성을 줄입니다.
- **데이터 공유 기술:**
  - **얕은 복제 (Shallow Clones) vs. 깊은 복제 (Deep Clones):** 얕은 복제는 메타데이터만 복사하고 실제 데이터는 공유하며, 깊은 복제는 데이터까지 모두 복사합니다.

- 데이터셋 직접 공유 (Direct Data Sharing): 데이터셋을 직접 공유하여 복제를 피합니다.
- 페더레이션 (Federation): 여러 데이터셋에 걸쳐 접근할 수 있도록 합니다.
- 뷰 (Views) 및 구체화된 뷰 (Materialized Views): 뷰를 통해 데이터를 추상화하고, 구체화된 뷰는 쿼리 결과를 캐시하여 성능을 향상시키고 비용을 절감합니다.
- 델타 타임 트래블 (Delta Time Travel): 데이터셋의 과거 버전에 접근하여 불필요한 백업 복사본을 줄입니다.

### 3. 데이터 제품에 대한 접근 민주화 (Democratize access to the data products):

- 비즈니스 사용자와 데이터 팀이 레이크하우스 내 데이터에 쉽게 접근할 수 있도록 하여 데이터 기반 의사결정을 촉진합니다.
- 중앙 집중식에서 분산형 의사결정으로 전환하여 조직 전체의 사람들이 필요한 데이터에 접근하고 더 나은 결정을 내릴 수 있도록 지원합니다.
- 적절한 도구와 가드레일(거버넌스)을 통해 올바른 데이터에 접근할 수 있도록 보장합니다.

### 4. 조직 전체의 데이터 거버넌스 전략 채택 (Adopt organization-wide data governance strategy):

- 데이터 시스템 전반에 걸쳐 일관된 거버넌스 규칙을 적용하여 데이터의 일관성, 완전성, 유효성을 보장합니다.
- 거버넌스의 세 가지 핵심 요소:
  - 데이터 품질 (Data Quality): 제약 조건, 적시성(SLA) 등을 통해 데이터의 정확성과 유효성을 보장합니다.
  - 데이터 카탈로그 (Data Catalog): 사용 가능한 데이터, 메타데이터(출처, 품질, 신뢰성), 데이터 계보(Lineage)를 식별하는 데 도움을 줍니다.
  - 접근 제어 (Access Control): 행 및 열 수준 접근 제어, 데이터 마스킹 및 익명화(PII), 감사 로그(Audit Logs)를 통해 데이터 보안을 유지합니다.

### 5. 개방형 인터페이스 및 개방형 형식 사용 권장 (Encourage the use of open interfaces and open formats):

- 상호운용성 (Interoperability): 타사 도구 및 파트너 도구와의 호환성을 제공합니다.
- 벤더 종속성 (Vendor Lock-in) 방지: 특정 벤더의 독점 형식에 묶이는 것을 방지하여 미래에 더 유연하고 비용 효율적인 솔루션으로 전환할 수 있도록 합니다.
- 소유 비용 (Cost of Ownership) 절감: 저비용 클라우드 스토리지(예: Spark 프레임워크에서 저비용 디스크 사용)를 활용하고 고가의 독점 플랫폼을 피합니다.
- 스토리지 계층화: 빠르고 비싼 스토리지와 느리지만 저렴한 스토리지 옵션을 활용할 수 있습니다.

### 6. 확장 가능하게 구축하고 성능 및 비용 최적화 (Build to scale and optimize for performance and costs):

- **빅데이터 시스템:** 많은 데이터와 컴퓨팅 요구사항에 대응하기 위해 확장성을 고려해야 합니다.
- **수직적 확장 (Vertical Scaling) vs. 수평적 확장 (Horizontal Scaling):**
  - **수직적 확장 (Scale Up):** 더 큰 노드(VM)를 사용하여 컴퓨팅 또는 스토리지 용량을 늘립니다.
  - **수평적 확장 (Scale Out):** 더 많은 노드를 추가하여 컴퓨팅 또는 스토리지 용량을 늘립니다. 빅데이터에서는 일반적으로 수평적 확장이 선호됩니다.
- **가격 대비 성능 (Price-Performance):** 비즈니스 사용자의 성능 기대치(예: 데이터 접근 시간)를 이해하고, 그에 맞춰 파이프라인을 설계하여 불필요한 고비용 성능을 피합니다.
- **스토리지와 컴퓨팅 분리 (Separate Storage from Compute):**
  - 컴퓨팅과 스토리지가 결합된 시스템(예: Elasticsearch)은 데이터가 증가함에 따라 컴퓨팅 비용도 불필요하게 증가하여 매우 비효율적일 수 있습니다.
  - 스토리지는 컴퓨팅보다 훨씬 저렴하므로, 둘을 분리하여 스토리지를 무제한으로 확장하고 컴퓨팅은 필요할 때만 추가하여 비용을 최적화합니다.
- **ROI 극대화:** 파이프라인이 요구사항을 충족하면서도 효율적으로 작동하여 비즈니스에 좋은 가치를 제공하도록 합니다.

## 0.4 데이터 파이프라인 구성 요소

데이터 파이프라인은 여러 핵심 구성 요소들이 유기적으로 결합하여 데이터를 처리하고 이동시킵니다.

### (a) 소스 및 대상 (Sources and Destinations):

- **소스:** 파이프라인으로 데이터가 유입되는 지점입니다. 구조화된 데이터(관계형 DB), 반정형 데이터(JSON, XML), 비정형 데이터(로그, 이미지, 비디오) 등 다양한 유형과 형태의 데이터를 포함할 수 있습니다.
- **대상:** 파이프라인에서 처리된 데이터가 최종적으로 저장되거나 소비되는 지점입니다. 데이터 웨어하우스, 데이터 레이크, 대시보드, ML 모델, 다른 애플리케이션 등이 될 수 있습니다.

### (b) 데이터 흐름 그래프 (Graph):

- 데이터의 이동과 종속성을 시각적으로 표현한 것입니다. 각 노드는 데이터 처리 단계(예: 변환, 필터링)를 나타내고, 엣지는 데이터의 흐름을 나타냅니다.
- Paul의 데모에서 LakeFlow의 그래픽 인터페이스를 통해 이러한 그래프를 정의하고 시각화하는 방법을 볼 수 있습니다.

### (c) 저장소 (Storage):

- 파이프라인 내에서 데이터가 임시 또는 영구적으로 저장되는 위치입니다. 데이터 레이크, 데이터 웨어하우스, 클라우드 스토리지(S3, ADLS) 등이 사용될 수 있습니다.
- 효율적인 스토리지 관리는 비용과 성능에 직접적인 영향을 미칩니다.

### (d) ETL 또는 ELT (Extract, Transform, Load / Extract, Load, Transform):

- **ETL:** 데이터를 원본에서 추출(Extract)하고, 필요한 형태로 변환(Transform)한 다음, 대상 시스템에 로드(Load)하는 방식입니다. 스키마가 잘 정의되고 구조화된 데이터에 적합합니다.
- **ELT:** 데이터를 원본에서 추출(Extract)하고, 대상 시스템에 먼저 로드(Load)한 다음, 대상 시스템 내에서 변환(Transform)하는 방식입니다. 비정형 또는 반정형 데이터, 대규모 데이터 처리에 유연하며, 클라우드 데이터 웨어하우스 환경에서 많이 사용됩니다.
- 어떤 방식을 사용할지는 데이터 유형, 볼륨, 변환 복잡성, 대상 시스템의 기능에 따라 달라집니다.

### (e) 워크플로우 (Workflow):

- 파이프라인의 각 단계가 실행되는 순서와 스케줄링, 오케스트레이션을 제어하는 메커니즘입니다.
- Airflow, Databricks Jobs와 같은 도구를 사용하여 워크플로우를 정의하고 자동화할 수 있습니다.

### (f) 모니터링 (Monitoring):

- 파이프라인의 정상 작동 여부를 지속적으로 확인하는 과정입니다.



- **목적:** 데이터 드리프트(Data Drift)와 같은 문제를 조기에 감지하고, 파이프라인의 견고성(Robustness)을 확보하며, 성능 지표(Metrics) 및 핵심 성과 지표(KPI)를 추적합니다.
- **예시:** 판매 데이터가 갑자기 급감하는 경우, 이는 파이프라인에 문제가 발생하여 데이터를 올바르게 보고하지 못하고 있음을 나타내는 비즈니스 지표가 될 수 있습니다.
- **견고성:** 문제가 발생했을 때 자동으로 재시도(Retry)하여 파이프라인이 계속 작동하도록 돕습니다.

## 0.5 데이터 유효성 검사 및 품질 관리

데이터 파이프라인에서 데이터 품질을 유지하는 것은 매우 중요합니다. 유효성 검사는 데이터가 파이프라인을 통과하면서 품질이 저하되지 않고 오히려 향상되도록 돕습니다.

### 0.5.1 데이터 검사 유형 (Types of Data Checks)

데이터에 발생할 수 있는 일반적인 문제들을 식별하기 위한 검사 유형은 다음과 같습니다.

- **누락된 정보 (Missing Information):** 필수 필드가 비어 있는 경우.
- **불완전한 정보 (Incomplete Information):** 데이터가 부분적으로만 존재하는 경우.
- **스키마 불일치 (Schema Mismatch):** 예상 스키마와 실제 데이터의 스키마가 다른 경우 (예: 정수여야 할 값이 실수로 들어옴).
- **상이한 형식 또는 데이터 유형 (Differing Formats or Data Types):** 날짜 형식이 다르거나, 문자열이 숫자로 들어오는 경우.
- **사용자 오류 (User Errors):** 데이터 생산자가 데이터를 잘못 입력한 경우.

### 0.5.2 오류 처리 모드 (Error Handling Modes)

Spark 플랫폼에서는 잘못된 데이터를 처리하는 다양한 모드를 제공합니다.

Table 3: 잘못된 데이터 처리 모드

| 모드                      | 설명                                                                                                   |
|-------------------------|------------------------------------------------------------------------------------------------------|
| <b>Bad Records Path</b> | 문제가 있는 레코드의 오류 설명을 특정 경로의 파일에 기록합니다. 이를 통해 나중에 오류를 분석하고 파이프라인을 수정할 수 있습니다.                           |
| <b>Drop formed</b>      | 손상된 레코드를 무시하고 데이터 흐름에서 제외합니다. 가장 엄격한 모드 중 하나입니다.                                                     |
| <b>Permissive</b>       | 가장 유연한 모드로, 잘못된 데이터를 허용하고 기록합니다. 누락된 컬럼에는 null 값을 넣고, 손상된 레코드는 corrupt라는 컬럼에 추가하여 나중에 분석할 수 있도록 합니다. |

Bad Records Path 설정 예시:

```
1 df = spark.read.option("badRecordsPath", "/tmp/bad_records_path").json(
    "path/to/data.json")
```

Listing 1: Bad Records Path 설정 예시

이 설정을 통해 파이프라인의 어느 단계에서든 손상된 레코드 정보를 기록하고 분석할 수 있습니다.

### 0.5.3 데이터 품질 향상 전략 (Strategies for Improving Data Quality)

데이터의 품질을 높이기 위해 데이터를 수정하는 다양한 전략이 있습니다.

- **중복 레코드 제거 (Dropping Duplicate Records):**
  - 특정 컬럼(예: ID, 선호 색상)을 기준으로 중복된 레코드를 제거합니다.
- **누락된 값 처리 (Handling Missing Values):**
  - **레코드 삭제 (Drop Records):** 하나 이상의 컬럼에 값이 누락된 레코드를 완전히 삭제합니다.
  - **플레이스홀더 추가 (Add Placeholder):** 누락된 값 대신 특정 값(예: -1)을 삽입하여 나중에 식별하고 수정할 수 있도록 합니다.
  - **값 대체 (Impute Values):** 누락된 값에 대해 최적의 추정치를 자동으로 채워 넣습니다.
    - \* **단순 대체:** 평균, 중앙값, 최빈값 등 통계적 방법을 사용합니다 (예: 온도가 누락되면 68도로, 바람이 누락되면 6mph로 설정).
    - \* **고급 대체:** 머신러닝 모델이나 복잡한 함수를 사용하여 더 정교하게 값을 추정합니다.
- **데이터 조작 함수 (Data Manipulation Functions):**
  - **Explode, Pivot, Cube, Rollup:** 데이터를 재구성하고 집계하는 데 사용되는 함수들입니다.

### 0.5.4 데이터 품질의 중요성 (Importance of Data Quality)

데이터 품질은 비즈니스 운영 및 의사결정에 광범위한 영향을 미칩니다.

- **정확성 (Accuracy):** 데이터가 올바르게 유효한가?
- **일관성 (Consistency):** 데이터 내의 관계, 이름, ID 등이 일관적인가?
- **완전성 (Completeness):** 누락되거나 부분적인 데이터가 없는가?
- **유효성 (Validity):** 데이터가 올바른 스키마를 따르는가?
- **적시성 (Timeliness):** 데이터가 신선하고 최신 정보인가? (예: 일주일 이상 된 데이터는 유효하지 않음)
- **무결성 (Integrity):** 데이터의 정확성이 유지되는가?

**Garbage In, Garbage Out (GIGO):** 잘못된 데이터가 입력되면 잘못된 결과가 나옵니다.

- **신뢰할 수 있는 의사결정:** 잘못된 데이터에 기반한 비즈니스 의사결정은 치명적인 결과를 초래할 수 있습니다.
- **분석 및 AI:** AI 모델의 편향(Bias)이나 부정확한 예측은 데이터 품질 문제에서 비롯될 수 있습니다.

- **운영 효율성:** 잘못된 데이터로 인한 문제는 파이프라인 속도를 늦추고 오프라인 상태로 만들 수 있습니다.
- **규정 준수 및 보안:** 데이터 품질은 규제 준수 및 보안에도 영향을 미칩니다.
- **고객 신뢰:** 고객이 잘못된 데이터로 인해 손실을 입으면 비즈니스에 대한 신뢰를 잃게 됩니다.

#### 데이터 품질 향상 예시: 생년월일 형식 변환

- **문제:** 테이블에 생년월일(birth date)이 타임스탬프(timestamp) 형식으로 저장되어 있어, 월/일/년 정보만 필요한 비즈니스 사용자에게는 유용성이 떨어집니다.
- **해결:** 'date\_of\_birth' , 'birthdate' 'date' / / .

```
1 SELECT
2     birth_date,
3     CAST(birth_date AS DATE) AS date_of_birth
4 FROM
5     your_table;
```

Listing 2: 생년월일 형식 변환 예시

이러한 변환은 데이터를 비즈니스 사용자에게 더 유용하게 만듭니다.

## 0.6 데이터 거버넌스

데이터 거버넌스는 데이터 파이프라인 전반에 걸쳐 모든 데이터 자산(구조화된 데이터, 반정형 데이터, 비정형 데이터, 모델, 대시보드, 서비스, 제품 포함)에 대한 통제 시스템을 구축하는 것입니다.

### 0.6.1 거버넌스의 범위

- **접근 제어 (Access Controls):** 누가 어떤 데이터에 어떤 수준의 접근 권한을 가지는가?
- **데이터 계보 (Lineage):** 데이터가 어디에서 왔고, 어떤 변환 과정을 거쳤으며, 현재 상태에 이르기까지 어떤 일이 있었는가?
- **데이터 발견 가능성 (Discoverability):** 데이터 카탈로그를 통해 유용한 데이터와 데이터 제품을 쉽게 찾을 수 있는가?
- **모니터링 (Monitoring):** 데이터 파이프라인의 흐름을 지속적으로 감시하는가?
- **감사 (Auditing):** 누가 데이터에 접근하고 수정했는지 기록하고 분석하는가?
- **공유 (Sharing):** 적절한 접근 제어 하에 데이터를 공유할 수 있는가?

### 0.6.2 데이터 거버넌스의 중요성

- **데이터 품질 및 일관성 보장:** 데이터가 정확하고 신뢰할 수 있도록 합니다.
- **보안 및 규정 준수:** 데이터 보호 및 법적 요구사항을 충족합니다.
- **운영 효율성:** 데이터 관련 문제로 인한 지연을 줄이고 워크플로우를 간소화합니다.
- **데이터에 대한 신뢰 증대:** 비즈니스 커뮤니티와 데이터 과학자들이 데이터에 대한 확신을 가질 수 있도록 합니다.
- **엔지니어링 워크플로우 간소화:** 데이터 파이프라인의 그래픽 표현을 통해 소스, 대상 및 그 사이의 과정을 더 잘 이해할 수 있습니다.

## 0.7 확장성, 탄력성, 신뢰성, 가용성

이 네 가지 개념은 고성능 데이터 시스템을 구축하는 데 필수적입니다. 확장성과 탄력성은 변화하는 요구사항에 따라 가용성과 성능을 향상시키는 데 도움을 줍니다.

### 0.7.1 확장성 (Scalability)

시스템이 추가 리소스(컴퓨팅, 스토리지)를 추가했을 때 워크로드와 처리량을 증가시킬 수 있는 능력입니다.

- **예시:** 데이터 저장 용량을 두 배로 늘렸을 때, 시스템의 데이터 처리 능력도 그에 비례하여 증가해야 합니다.
- **수직적 확장 (Scale Up):** 더 큰 노드(예: 더 강력한 CPU, 더 많은 RAM을 가진 서버)를 사용하여 용량을 늘리는 방식입니다. 한계가 있으며 비용이 많이 들 수 있습니다.
- **수평적 확장 (Scale Out):** 더 많은 노드(일반적으로 작고 저렴한 노드)를 추가하여 용량을 늘리는 방식입니다. 빅데이터 시스템에서 선호되며, 자동 확장이 용이합니다.

### 0.7.2 탄력성 (Elasticity)

시스템이 워크로드 변화에 동적으로 적응하여 리소스를 자동으로 늘리거나 줄일 수 있는 능력입니다.

- **예시:** 데이터 처리량이 급증할 때 자동으로 컴퓨팅 리소스를 늘리고, 처리량이 감소하면 다시 줄여서 불필요한 비용 지출을 막습니다.
- **Databricks 서버리스 프레임워크:** 사용자가 사용하는 만큼만 비용을 지불하고, 필요에 따라 자동으로 리소스가 확장/축소되는 좋은 예시입니다. 사용하지 않을 때는 리소스가 0으로 줄어들어 비용이 발생하지 않습니다.

### 0.7.3 신뢰성 (Reliability)

시스템이 오류 없이 올바른 동작을 지속적으로 수행하는 능력입니다. 데이터 파이프라인의 경우, 정확한 데이터 출력을 보장하는 것을 의미합니다.

### 0.7.4 가용성 (Availability)

시스템이 사용자가 필요할 때 언제든지 접근하여 사용할 수 있는 상태를 유지하는 능력입니다.

- **비즈니스 요구사항:** 시스템이 항상 가용해야 하는지, 아니면 유지보수를 위해 주기적으로 다운될 수 있는지 등은 비기능적 요구사항에서 명확히 정의되어야 합니다.
- **예시:** '5-나인즈(99.999%)' 가용성은 매우 높은 수준으로, 연간 약 5분 정도의 다운타임만 허용합니다. 이는 매우 높은 비용을 수반하므로, 비즈니스 중요도에 따라 적절한 수준을 결정해야 합니다.

## 0.8 LakeFlow 및 선언적 파이프라인 (Declarative Pipelines)

데이터 파이프라인 구축은 데이터 조작 코드 자체보다 탄력성, 확장성, 데이터 정확성 등 운영적 측면에서 더 많은 어려움을 겪습니다. Databricks의 LakeFlow 선언적 파이프라인 (이전 명칭: Delta Live Tables, DLT)은 이러한 복잡성을 해결하기 위해 고안되었습니다.

### 0.8.1 선언적 파이프라인의 필요성 및 이점

- **운영 복잡성 해결:** 파이프라인의 확장, 예상치 못한 데이터 유입 처리, 배치 및 스트리밍 상황 통합 등 복잡한 운영 문제를 자동화합니다.
- **간단한 작성 (Simple Authoring):** 단 한 줄의 SQL 또는 Python 문장으로 강력한 파이프라인을 정의할 수 있습니다. 엔진이 모든 최적화, 배치/스트리밍 통합을 자동으로 처리합니다.
- **비용 효율성 (Cost-Efficient):** 엔진이 효율적으로 실행되고, 필요에 따라 리소스를 확장/축소하며, 서버리스 컴퓨팅을 활용하여 비용을 최적화합니다.
- **재사용 가능한 패턴:** 데이터 엔지니어가 반복적으로 작성해야 했던 일반적인 운영 패턴 (예: 데이터 품질 검사, 스키마 변경 처리)을 엔진에 내장하여 개발 시간을 단축합니다.
- **전체 파이프라인 최적화:** 개별 코드 조각이 아닌 전체 파이프라인의 종속성을 분석하여 최적의 실행 계획을 수립하고 병렬 처리를 가능하게 합니다.

### 0.8.2 LakeFlow 아키텍처

LakeFlow는 소스 데이터 연결(예: Autoloader), 선언적 파이프라인 엔진, 그리고 다양한 출력(Delta 테이블)으로 구성됩니다.

- **소스 데이터 (Source Data):** Autoloader, Kafka 등 다양한 소스에서 데이터를 읽어옵니다.
- **선언적 파이프라인 (Declarative Pipelines):** 엔진이 파이프라인을 실행하는 데 사용하는 선언적 코드 집합입니다. Python 또는 SQL로 작성할 수 있습니다.
- **오케스트레이션 (Orchestration):** LakeFlow는 전체 파이프라인을 분석하여 데이터 흐름, 종속성, 병렬 실행 가능성을 파악하고 최적의 실행 계획을 수립합니다.

### 0.8.3 주요 구성 요소

선언적 파이프라인의 핵심 빌딩 블록은 스트리밍 테이블과 구체화된 뷰입니다.

#### (a) 스트리밍 테이블 (Streaming Tables):

- 증분 데이터 처리를 위해 설계되었습니다. 이미 처리된 데이터는 다시 처리하지 않아 낮은 지연 시간과 효율적인 스케일링을 제공합니다.
- 주로 Bronze 레이어에서 Silver 레이어로 데이터를 이동하는 데 사용됩니다 (원시 데이터 -> 정제된 데이터).

- **변경 데이터 캡처 (Change Data Capture, CDC):** 삽입 (Insert), 업데이트 (Update), 삭제 (Delete) 작업을 자동화하여 처리합니다.
- **스키마 진화 (Schema Evolution):** 스키마 변경에 대한 다양한 처리 전략을 제공합니다 (아래에서 상세 설명).

(b) **구체화된 뷰 (Materialized Views):**

- 쿼리 결과를 미리 계산하여 캐시해 둔 뷰입니다. 이를 통해 쿼리 성능을 크게 향상시키고 반복적인 계산 비용을 절감할 수 있습니다.
- Gold 레이어에서 집계된 데이터를 생성하는 데 주로 사용됩니다.
- 엔진이 증분 업데이트를 자동으로 처리하므로, 사용자는 뷰를 정의하는 것만으로도 거의 실시간에 가까운 데이터 업데이트를 얻을 수 있습니다.

#### 0.8.4 스키마 변경 관리 (Schema Evolution)

데이터 소스의 스키마는 시간이 지남에 따라 변경될 수 있습니다. 선언적 파이프라인은 이러한 변경에 대응하는 유연한 전략을 제공합니다.

- **스키마 강제 (Enforce Schema):** 스키마가 예상과 다를 경우 파이프라인을 중단시킵니다. 이는 데이터 품질이 매우 중요하고 스키마 오류가 심각한 문제로 간주될 때 사용됩니다. 파이프라인은 상태를 유지하므로, 스키마를 수정한 후 다시 시작할 수 있습니다.
- **스키마 진화 (Evolve Schema):** 스키마 변경을 허용하고, 새로운 컬럼 등을 자동으로 통합합니다. 이는 소스 데이터가 새로운 정보를 제공할 것으로 예상될 때 유용합니다. 다양한 병합 전략을 지정할 수 있습니다.

#### 0.8.5 데이터 품질 관리 (Data Quality Management with Expectations)

선언적 파이프라인은 데이터 품질 검사를 파이프라인 정의에 직접 통합할 수 있도록 '기대치 (Expectations)' 개념을 제공합니다. 이를 통해 잘못된 데이터에 대한 처리 방식을 선언적으로 정의할 수 있습니다.

- **기대치 설정 (Setting Expectations):**
  - 데이터에 대한 특정 조건을 정의합니다 (예: 고객 연령이 0에서 120 사이여야 함).
  - 이 조건은 간단한 null 검사부터 복잡한 함수 호출까지 다양할 수 있습니다.
- **잘못된 데이터 처리 선택지:**
  - (a) **데이터 기록 및 추적 (Write and Note):** 잘못된 데이터를 허용하되, 해당 데이터가 잘못되었음을 표시하여 나중에 필터링하거나 분석할 수 있도록 합니다. (예: 웹사이트 클릭 데이터처럼 높은 정확도가 필요 없는 경우)
  - (b) **데이터 삭제 (Drop Data):** 조건에 맞지 않는 레코드를 파이프라인에서 완전히 삭제합니다.
  - (c) **파이프라인 실패 (Fail Pipeline):** 조건이 충족되지 않으면 파이프라인을 중단시킵니다. (예: 금융 정보와 같이 데이터 정확성이 매우 중요한 경우)



- **데이터 격리 (Quarantining Data):**

- 기대치를 통과하지 못한 레코드를 별도의 '격리 테이블(Quarantine Table)'에 저장할 수 있습니다.
- 이를 통해 파이프라인은 계속 실행되면서도, 잘못된 데이터를 나중에 수정하고 재주입할 수 있는 기회를 제공합니다. 이는 파이프라인의 연속성을 유지하면서 데이터 품질 문제를 관리하는 일반적인 패턴입니다.

## 실습/코드/오류와 해결

### 0.9 Paul의 LakeFlow 선언적 파이프라인 데모 요약

Paul은 LakeFlow 선언적 파이프라인의 실제 적용 사례를 데모를 통해 보여주었습니다. 이 데모는 5억 개 이상의 행을 가진 대규모 CDC(Change Data Capture) 데이터를 처리하는 파이프라인을 구축하는 과정을 시뮬레이션했습니다.

#### 0.9.1 데모 시나리오

- **데이터 생성:** 5억 개 이상의 행을 가진 Parquet 파일 형태의 CDC 데이터를 생성했습니다. 이 데이터는 삽입(Insert), 업데이트(Update), 삭제>Delete) 작업을 포함합니다.
- **파이프라인 구성:**
  - **Bronze Layer (원시 데이터):** Autoloader를 사용하여 원시 데이터를 수집합니다. Python 코드에서는 `@dlt.table` 데코레이터를 사용하여 이 단계를 정의합니다.
  - **Cleaned View (데이터 정제):** Bronze 테이블에서 읽어와 데이터 유형 캐스팅 등 간단한 정제 작업을 수행하는 뷰를 정의합니다. 이는 메모리에서 실행됩니다.
  - **Silver Layer (정제된 데이터):** 정제된 뷰를 소스로 사용하여 `apply_changes CDC` .
    - `apply_changes` , , , , .
    - 이를 통해 삽입, 업데이트, 삭제 로직을 수동으로 코딩할 필요 없이 선언적으로 정의할 수 있습니다.
    - 스키마가 잘못된 경우 파이프라인이 실패하도록 설정하여 데이터 품질을 엄격하게 관리했습니다.

#### 7. 파이프라인 실행 및 모니터링:

- Databricks UI의 LakeFlow 디자이너를 통해 파이프라인의 DAG(Directed Acyclic Graph)를 시각적으로 확인하고 실행할 수 있습니다.
- **전체 새로고침 (Full Refresh):** 모든 테이블을 처음부터 다시 빌드하는 옵션입니다. 이 경우 파이프라인은 상태를 재설정하고 모든 데이터를 재처리합니다.
- **증분 업데이트 (Incremental Update):** 새로운 데이터만 찾아 처리합니다. 데이터가 없는 경우 매우 빠르게 완료됩니다.
- **선택적 테이블 새로고침:** 파이프라인 내의 특정 테이블만 선택하여 새로고침할 수 있습니다.
- **이벤트 로그 (Event Log):** 파이프라인의 모든 작업(시작, 리소스 대기, 초기화, 테이블 생성, 실행 등)이 기록되어 문제 발생 시 디버깅에 활용됩니다. 이 로그는 Delta 테이블에 저장되어 완전한 가시성을 제공합니다.
- **쿼리 기록 (Query History):** 엔진이 자동으로 생성하고 실행한 SQL 쿼리 및 실행 계획을 확인할 수 있습니다. 이를 통해 성능 분석 및 최적화에 필요한 정보를 얻을 수 있습니다.

8. **API를 통한 제어:** 실제 프로덕션 환경에서는 UI를 통한 수동 실행 대신 API를 통해 파이프라인을 다양한 모드(전체 새로고침, 특정 테이블 새로고침 등)로 제어하는 것이 일반적입니다.

### 0.9.2 데이터 품질 관리 (Expectations) 데모 (문서 기반)

Paul은 문서 기반으로 선언적 파이프라인의 데이터 품질 관리 기능인 '기대치(Expectations)'를 설명했습니다.

- **기대치 정의:** `expect` 키워드를 사용하여 데이터에 대한 조건을 선언합니다(예: `expect("valid customer age" = "age BETWEEN 0 AND 120")`).
- **처리 방식 선택:**
  - **KEEP:** 조건에 맞지 않아도 데이터를 유지하고, 문제가 있음을 기록합니다. (예: `expect_or_drop("validcustomerage" = "ageBETWEEN0AND120")`)
  - **DROP:** 조건에 맞지 않는 레코드를 삭제합니다.
  - **FAIL:** 조건에 맞지 않으면 파이프라인을 중단시킵니다.
- **데이터 격리 (Quarantine):** 기대치를 통과하지 못한 레코드를 별도의 테이블에 저장하여 나중에 수정하고 재주입할 수 있도록 합니다. 이는 파이프라인의 흐름을 유지하면서 데이터 품질 문제를 관리하는 강력한 방법입니다.

## 0.10 과제 3 오류와 해결: 체크포인트 위치 문제

강의 중 과제 3에서 발생한 일반적인 오류와 그 해결책에 대한 질문이 있었습니다.

### 0.10.1 증상

스트리밍 데이터 수집 파이프라인을 실행할 때, 체크포인트(checkpoint) 위치와 관련된 오류가 발생하여 파이프라인이 정상적으로 작동하지 않습니다. 특히 DBFS 경로 대신 Unity Catalog 볼륨을 사용하려 할 때 문제가 발생했습니다.

### 0.10.2 원인 가설

- **잘못된 체크포인트 경로 형식:** Unity Catalog 볼륨을 사용할 때 경로 형식이 DBFS와 다를 수 있습니다.
- **체크포인트 데이터 충돌:** 이전에 생성된 체크포인트 데이터가 남아 있어 새로운 스트림 실행과 충돌할 수 있습니다. 스트리밍 쿼리는 상태를 유지하므로, 이전 실행의 체크포인트가 남아 있으면 문제가 발생할 수 있습니다.

### 0.10.3 수정 방법

해결책은 Unity Catalog 볼륨을 올바른 형식으로 지정하고, 매번 스트림을 시작하기 전에 이전 체크포인트를 삭제하는 것입니다.

1. **Unity Catalog 볼륨 경로 사용:** 체크포인트 위치로 Unity Catalog 볼륨을 지정할 때는 `/Volumes/<카탈로그>/<스키마>/<볼륨>/<경로>` 형식을 사용해야 합니다.

2. **이전 체크포인트 삭제:** 스트리밍 쿼리를 다시 시작할 때마다 이전 체크포인트 디렉토리를 삭제하여 상태 충돌을 방지합니다.

**과제 3 체크포인트 오류 해결 코드 예시:**

```

1 # 1. 체크포인트경로정의 (Unity Catalog 볼륨사용 )
2 checkpoint_path = "/Volumes/your_catalog/your_schema/your_volume/
   checkpoints/assignment3_stream"
3
4 # 2. 이전체크포인트삭제매번 ( 실행전 )
5 dbutils.fs.rm(checkpoint_path, True) # True 재귀적으로삭제를의미
6
7 # 3. 스트리밍쿼리실행
8 (spark.readStream
9   .format("cloudFiles")
10  .option("cloudFiles.format", "json")
11  .option("cloudFiles.schemaLocation", "/Volumes/your_catalog/your_schema/
   your_volume/schemas/assignment3_stream") # 스키마위치도볼륨으로지
   정
12  .load("/Volumes/your_catalog/your_schema/your_volume/data/input_stream")
   # 입력데이터위치도볼륨으로지
   정
13  .writeStream
14  .format("delta")
15  .outputMode("append")
16  .option("checkpointLocation", checkpoint_path) # 올바른체크포인트경로사용
17  .toTable("your_catalog.your_schema.output_table") # 출력테이블도볼륨으로지정
18 )

```

Listing 3: 체크포인트 오류 해결 코드

**체크포인트의 중요성:** 체크포인트는 스트리밍 쿼리의 상태를 저장하여 장애 발생 시 중단된 지점부터 복구할 수 있도록 합니다. 따라서 프로덕션 환경에서는 체크포인트를 삭제하는 대신, 파이프라인의 재시작 로직이 기존 체크포인트를 활용하도록 설계해야 합니다. 과제나 개발 단계에서는 디버깅을 위해 삭제하는 것이 유용할 수 있습니다.

## 체크리스트

이 문서를 통해 학습한 내용을 점검하고, 데이터 파이프라인 설계 및 운영 시 고려해야 할 사항들을 확인하는 체크리스트입니다.

### 1. 요구사항 정의:

- 기능적 요구사항(무엇을 할 것인가)이 명확히 정의되었는가?
- 비기능적 요구사항(어떻게 할 것인가: 성능, 가용성, 확장성 등)이 비즈니스와 합의되었는가?
- 특히 가용성(예: 5-나인즈) 및 지연 시간에 대한 기대치가 현실적인가?

### 2. 파이프라인 설계 원칙:

- 데이터가 신뢰할 수 있는 '제품'으로 제공되는가? (메달리온 아키텍처 고려)
- 데이터 사일로를 제거하고 데이터 이동을 최소화하는가? (얇은 복제, 델타 타임 트래블 등 활용)
- 데이터 접근이 민주화되어 조직 전체에서 활용 가능한가?
- 조직 전체의 일관된 데이터 거버넌스 전략이 적용되었는가? (품질, 카탈로그, 접근 제어)
- 개방형 인터페이스 및 형식을 사용하여 벤더 종속성을 피하고 비용을 최적화하는가?
- 확장 가능하게 구축되었으며, 성능과 비용이 최적화되었는가? (스토리지/컴퓨팅 분리)

### 3. 데이터 품질 및 유효성 검사:

- 누락, 불완전, 스키마 불일치 등 일반적인 데이터 오류에 대한 검사 로직이 포함되었는가?
- 잘못된 데이터 처리 모드(Drop Malformed, Permissive, Bad Records Path)가 적절히 사용되었는가?
- 중복 제거, 누락 값 대체(Imputation) 등 데이터 품질 향상 전략이 적용되었는가?
- 데이터 품질 속성(정확성, 일관성, 완전성, 적시성 등)이 정의되고 측정되는가?

### 4. 데이터 거버넌스:

- 접근 제어(행/열 수준, 마스킹) 및 감사 로그가 구현되었는가?
- 데이터 계보(Lineage) 추적이 가능한가?
- 데이터 카탈로그를 통해 데이터 발견 가능성이 확보되었는가?

### 5. 확장성 및 탄력성:

- 워크로드 변화에 따라 리소스가 자동으로 확장/축소되는 탄력적인 구조인가? (서버리스 컴퓨팅 활용)
- 수평적 확장을 통해 대규모 데이터 처리에 대응하는가?

### 6. 자동화 및 모니터링:

- 파이프라인의 모든 단계가 자동화되어 수동 개입을 최소화하는가?
- 파이프라인의 상태, 성능, 데이터 품질을 모니터링하는 시스템이 구축되었는가?
- 데이터 드리프트 및 비즈니스 지표를 통한 이상 감지가 가능한가?

### 7. LakeFlow/선언적 파이프라인 활용:

- 복잡한 운영 로직(CDC, 스키마 진화, 데이터 품질 기대치)을 선언적으로 정의하여 개발 및 유지보수 비용을 절감하는가?
- 스트리밍 테이블과 구체화된 뷰를 활용하여 효율적인 증분 처리를 구현하는가?

## FAQ (자주 묻는 질문)

**Q: 데이터 파이프라인 운영화가 정확히 무엇인가요?**

**A:** 데이터 파이프라인 운영화는 데이터 과학자가 개발한 초기 분석 모델이나 프로토타입을 실제 비즈니스 환경에서 지속적으로 실행되고, 신뢰할 수 있으며, 자동화된 시스템으로 전환하는 과정입니다. 이는 단순히 코드를 실행하는 것을 넘어, 데이터 품질, 보안, 확장성 등 다양한 운영적 측면을 고려하여 비즈니스 가치를 지속적으로 창출하는 것을 목표로 합니다.

**Q: 기능적 요구사항과 비기능적 요구사항의 차이점은 무엇인가요?**

**A:** 기능적 요구사항은 시스템이 '무엇을' 해야 하는지(예: 고객 주문 처리, 보고서 생성)를 정의하는 반면, 비기능적 요구사항은 시스템이 '어떻게' 작동해야 하는지(예: 3초 이내 응답, 99.999% 가용성, 데이터 암호화)를 정의합니다. 둘 다 중요하지만, 비기능적 요구사항은 종종 간과되어 시스템의 실패로 이어질 수 있습니다.

**Q: 데이터 사일로를 제거하고 데이터 이동을 최소화해야 하는 이유는 무엇인가요?**

**A:** 데이터를 불필요하게 복제하거나 이동하는 것은 데이터 불일치, 품질 저하, 지연 시간 증가, 그리고 운영 비용 상승을 초래합니다. 데이터 사일로는 데이터가 고립되어 조직 전체에서 활용되지 못하게 합니다. 데이터 공유 기술(얇은 복제, 뷰, 델타 타임 트래블)을 활용하여 이러한 문제를 해결하고 효율성을 높일 수 있습니다.

**Q: LakeFlow 선언적 파이프라인이 기존 스트리밍 코드보다 어떤 이점이 있나요?**

**A:** 선언적 파이프라인은 데이터 엔지니어가 수동으로 코딩해야 했던 복잡한 운영 로직(예: CDC, 스키마 진화, 오류 처리, 배치/스트리밍 통합, 리소스 스케일링)을 엔진이 자동으로 처리하도록 합니다. 이를 통해 개발자는 비즈니스 로직에만 집중할 수 있고, 파이프라인은 더 견고하고, 확장 가능하며, 비용 효율적으로 운영됩니다.

**Q: 데이터 품질 기대치(Expectations)는 어떻게 작동하며, 왜 중요한가요?**

**A:** 데이터 품질 기대치는 파이프라인 내에서 데이터에 대한 특정 조건을 선언적으로 정의하는 기능입니다. 예를 들어, '고객 연령은 0에서 120 사이여야 한다'와 같은 조건을 설정할 수 있습니다. 이 조건이 충족되지 않을 경우, 데이터를 삭제하거나, 파이프라인을 중단시키거나, 또는 별도의 격리 테이블에 저장하여 나중에 수정할 수 있습니다. 이는 잘못된 데이터가 하위 시스템으로 흘러가는 것을 방지하고 데이터 신뢰성을 유지하는 데 매우 중요합니다.

**Q:** 과제에서 체크포인트 위치 오류가 발생했는데, 어떻게 해결해야 하나요?

**A:** 이 오류는 주로 Unity Catalog 볼륨을 체크포인트 위치로 사용할 때 경로 형식이 잘못되었거나, 이전 실행의 체크포인트 데이터가 남아 있어 발생합니다. 해결책은 Unity Catalog 볼륨 경로를 `/Volumes/<카탈로그>/<스키마>/<볼륨>/<경로>` 형식으로 올바르게 지정하고, 스트리밍 쿼리를 시작하기 전에 `dbutils.fs.rm(checkpoint_path, True)` .

## 빠르게 훑어보기 (1 페이지 요약)

### 요약

#### CSCI103 Lecture 7: 데이터 파이프라인 운영화 핵심 요약

1. **운영화의 목표:** 데이터 과학자의 프로토타입을 신뢰할 수 있는 자동화된 파이프라인으로 전환하여 비즈니스 의사결정 지원.

#### 2. 요구사항 정의:

- **기능적:** 시스템이 '무엇을' 할 것인가 (비즈니스 규칙, 인터페이스).
- **비기능적:** 시스템이 '어떻게' 작동할 것인가 (성능, 가용성, 확장성, 보안). **중요!**

#### 3. 핵심 원칙:

- **데이터 제품화:** 메달리온 아키텍처 (Raw → Curated → Business Ready)로 품질 향상.
- **사일로 제거/이동 최소화:** 복제 피하고 델타 타임 트래블, 뷰 등으로 효율성 증대.
- **접근 민주화:** 조직 전체의 데이터 기반 의사결정 지원.
- **거버넌스:** 데이터 품질, 카탈로그, 접근 제어로 신뢰성 확보.
- **개방형 형식:** 벤더 종속성 피하고 비용 절감.
- **확장/최적화:** 수평적 확장, 스토리지/컴퓨팅 분리로 비용 효율성 극대화.

4. **파이프라인 구성 요소:** 소스/대상, 그래프, 스토리지, ETL/ELT, 워크플로우, 모니터링.

#### 5. 데이터 품질 관리:

- **검사 유형:** 누락, 스키마 불일치, 형식 오류 등.
- **오류 처리:** Drop Malformed, Permissive, Bad Records Path.
- **향상 전략:** 중복 제거, 누락 값 대체 (Imputation).
- **중요성:** "Garbage In, Garbage Out" - 신뢰할 수 있는 의사결정의 기반.

#### 6. 확장성, 탄력성, 신뢰성, 가용성:

- **확장성:** 리소스 추가 시 처리량 증가 (수직 vs. 수평).
- **탄력성:** 워크로드 변화에 따른 리소스 자동 조절 (서버리스).
- **신뢰성:** 올바른 동작 지속.
- **가용성:** 필요 시 사용 가능 (5-나인즈).

#### 7. LakeFlow 선언적 파이프라인 (DLT):

- **이점:** 복잡한 운영 로직 (CDC, 스키마 진화, 오류 처리)을 선언적으로 자동화하여 개발 간소화, 비용 효율성 증대.
- **구성:** 스트리밍 테이블 (충분 처리), 구체화된 뷰 (성능 최적화).

- **데이터 품질 기대치:** 파이프라인 내에서 데이터 조건 정의 및 오류 처리 (기록, 삭제, 실패, 격리) 자동화.